Web Programming Review 1

Project: PennyWise – Student Expense Tracker

Batch 1: Shourya Chavan (24BCE0509)

Aarush Singh (24BCE0488)

Shambhavi (24BCE0514)

Prisha Rawat (24BCE2545)

Aishashwi Gupta (24BCB0088)

Project Scope: PennyWise v1.0 (Review 1)

For the first review, the project focuses on delivering a functional Minimum Viable Product (MVP). The primary goal is to demonstrate a robust frontend architecture, seamless React state management, component-based routing, and a responsive User Interface.

✅IN-SCOPE (Completed for Review 1)

Component Architecture & UI Layout: Fully styled responsive interface utilizing Tailwind CSS, ensuring mobile-first compatibility across different screen sizes.

State-Based Routing: Implementation of a Single Page Application (SPA) navigation system that switches between Dashboard, History, and Admin tabs seamlessly without browser reloads.

React State Management: Core CRUD (Create, Read, Delete) operations using React useState. The application successfully holds and manipulates an array of transaction objects in the Virtual DOM.

Dynamic Calculations: Real-time computation of "Total Balance," "Total Income," and "Total Expenses" using React's useMemo hook for performance optimization.

Form Validation Logic: JavaScript-driven safeguards on the "Add Transaction" form to prevent the submission of empty fields, invalid text, or negative/zero amounts.

Admin Dashboard (MVP): A read-only system administrator portal that securely tracks current local state capacity, global liquidity, and mock active user sessions.

✕OUT-OF-SCOPE (Deferred to Review 2)

To ensure manageable development phases, the following advanced features have been intentionally reserved for the final review:

Data Persistence (Database/Storage): Currently, data resets upon a browser refresh. Review 2 will integrate persistent storage (browser localStorage or a backend database) to save user sessions.

HTML5 Canvas Visualization: The dashboard currently displays a placeholder. Review 2 will feature a custom-coded JavaScript logic engine to draw dynamic pie/doughnut charts using the HTML5 <canvas> API.

Advanced Data Filtering: The transaction history currently displays all records. Categorical sorting and filtering (e.g., viewing only "Food" or "Travel" expenses) will be activated in the next phase.

Admin Write-Privileges: Adding new users or injecting global categories via the Admin Panel is currently locked and will be made functional in Review 2.

Main Code:

Following is the main code for the implementation of our project.

```
import React, { useState, useMemo } from 'react';
// --- INLINE SVG ICONS --- const IconPieChart = ({ className }) =>
(); const IconList = ({ className }) => (); const IconWallet = ({
className }) => (); const IconTrash2 = ({ className }) => (); const
IconPlusCircle = ({ className }) => (); const IconArrowUpRight = ({
className }) => (); const IconArrowDownRight = ({ className }) => ();
const IconUtensils = ({ className }) => (); const IconCar = ({
className }) => (); const IconBookOpen = ({ className }) => (); const
IconFilm = ({ className }) => (); const IconMoreHorizontal = ({
className }) => (); const IconShield = ({ className }) => (); const
IconDatabase = ({ className }) => ();
// --- MAIN APP COMPONENT --- export default function App() { // 1.
MVP State Management (No Local Storage for Review 1) const
[transactions, setTransactions] = useState([ { id: 1, title: 'Monthly
Allowance', amount: 5000, type: 'income', category: 'Allowance',
date: new Date().toISOString().split('T')[0] }, { id: 2, title: 'Mess
Bill', amount: 1200, type: 'expense', category: 'Food', date: new
Date().toISOString().split('T')[0] }, { id: 3, title: 'Auto to
Katpadi', amount: 150, type: 'expense', category: 'Travel', date: new
Date().toISOString().split('T')[0] }, { id: 4, title: 'React
Textbook', amount: 450, type: 'expense', category: 'Academics', date:
new Date().toISOString().split('T')[0] } ]);
const [currentTab, setCurrentTab] = useState('dashboard'); // Simple
Routing
// 2. Core Logic Functions const addTransaction = (newTxn) => {
setTransactions([newTxn, ...transactions]); };
const deleteTransaction = (id) => {
setTransactions(transactions.filter(txn => txn.id !== id)); };
// 3. Derived State Calculations const { totalBalance, totalIncome,
totalExpense } = useMemo(() => { let income = 0; let expense = 0;
transactions.forEach(txn => { if (txn.type === 'income') income +=
Number(txn.amount); if (txn.type === 'expense') expense +=
Number(txn.amount); }); return { totalBalance: income - expense,
totalIncome: income, totalExpense: expense }; }, [transactions]);
// 4. Render active view based on "Routing" const renderView = () =>
{ switch (currentTab) { case 'dashboard': return ( ); case 'history':
return ( ); case 'admin': return ( ); default: return ; } };
return (
```

```
{/* Navigation Header */}
PennyWise v1.0 (Review 1)
<button onClick={() => setCurrentTab('dashboard')} className={px-3
py-2 rounded-md text-sm font-medium transition-colors flex items-
center gap-2 ${currentTab === 'dashboard' ? 'bg-indigo-700 text-
white' : 'text-indigo-100 hover:bg-indigo-500'}} > Dashboard <button
onClick={() => setCurrentTab('history')} className={px-3 py-2
rounded-md text-sm font-medium transition-colors flex items-center
gap-2 ${currentTab === 'history' ? 'bg-indigo-700 text-white' :
'text-indigo-100 hover:bg-indigo-500'}} > History <button onClick={()
=> setCurrentTab('admin')} className={px-3 py-2 rounded-md text-sm
font-medium transition-colors flex items-center gap-2 ${currentTab
=== 'admin' ? 'bg-slate-800 text-amber-400' : 'text-indigo-100
hover:bg-indigo-500'}} > Admin
 {/* Main Content Area */}
  <main className="max-w-4xl mx-auto p-4 sm:p-6 lg:p-8 animate-fade-
in">
    {renderView()}
  </main>
</div>

); }
// --- DASHBOARD COMPONENT --- function Dashboard({ totalBalance,
totalIncome, totalExpense, onAddTransaction }) { return (
```

Student Dashboard

Track your campus expenses.

```
 {/* Balance Cards */}
  <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
    <div className="bg-white p-6 rounded-2xl shadow-sm border border-
gray-100 flex flex-col justify-center">
      <span className="text-gray-500 text-sm font-medium">Total
Balance</span>
```

```
      <span className={text-3xl font-bold mt-2 ${totalBalance < 0 ?
'text-red-500' : 'text-gray-900'}}>
        ₹{totalBalance.toLocaleString()}
      </span>
    </div>
    <div className="bg-emerald-50 p-6 rounded-2xl shadow-sm border
border-emerald-100 flex items-center justify-between">
      <div>
        <span className="text-emerald-700 text-sm font-
medium">Income</span>
        <div className="text-2xl font-bold text-emerald-600 mt-
2">₹{totalIncome.toLocaleString()}</div>
      </div>
      <div className="bg-emerald-200 p-3 rounded-full text-emerald-
700">
        <IconArrowUpRight className="w-6 h-6" />
      </div>
    </div>
    <div className="bg-rose-50 p-6 rounded-2xl shadow-sm border
border-rose-100 flex items-center justify-between">
      <div>
        <span className="text-rose-700 text-sm font-
medium">Expenses</span>
        <div className="text-2xl font-bold text-rose-600 mt-
2">₹{totalExpense.toLocaleString()}</div>
      </div>
      <div className="bg-rose-200 p-3 rounded-full text-rose-700">
        <IconArrowDownRight className="w-6 h-6" />
      </div>
    </div>
  </div>

  <div className="grid grid-cols-1 lg:grid-cols-2 gap-6 items-start">
    {/* HTML5 Canvas Chart - DISABLED FOR REVIEW 1 */}
    <div className="bg-white p-6 rounded-2xl shadow-sm border border-
gray-100 flex flex-col justify-center items-center text-center h-
[380px]">
      <h2 className="text-lg font-bold text-gray-800 mb-4 self-
```

```
start">Expense Breakdown</h2>
      <div className="flex flex-col items-center justify-center p-8
bg-gray-50 rounded-xl border-2 border-dashed border-gray-200 w-full
h-full">
        <IconPieChart className="w-12 h-12 text-gray-300 mb-3" />
        <p className="text-gray-600 font-medium">Visual Engine
Pending</p>
        <p className="text-gray-400 text-sm mt-2 max-w-[200px]">The
HTML5 Canvas rendering logic will be fully integrated for Review
2.</p>
      </div>
    </div>

    {/* Transaction Form */}
    <div className="bg-white p-6 rounded-2xl shadow-sm border border-
gray-100 h-[380px]">
      <h2 className="text-lg font-bold text-gray-800 mb-4">Add
Transaction</h2>
      <TransactionForm onAdd={onAddTransaction} />
    </div>
  </div>
</div>

); }
// --- TRANSACTION FORM COMPONENT --- function TransactionForm({
onAdd }) { const [title, setTitle] = useState(''); const [amount,
setAmount] = useState(''); const [type, setType] =
useState('expense'); const [category, setCategory] =
useState('Food'); const [date, setDate] = useState(new
Date().toISOString().split('T')[0]); const [error, setError] =
useState('');
const expenseCategories = ['Food', 'Travel', 'Academics',
'Entertainment', 'Other']; const incomeCategories = ['Allowance',
'Freelance', 'Prize Money', 'Other'];
const handleSubmit = (e) => { e.preventDefault(); setError('');
if (!title.trim() || !amount) {
  setError('Please fill in all required fields.');
  return;
```

```
}
if (Number(amount) <= 0) {
  setError('Amount must be greater than zero.');
  return;
}

const newTxn = {
  id: crypto.randomUUID(),
  title: title.trim(),
  amount: Number(amount),
  type,
  category,
  date
};

onAdd(newTxn);
setTitle('');
setAmount('');

};
return (
{error &&
{error}
}
 <div className="flex bg-gray-100 p-1 rounded-lg">
    <button type="button" onClick={() => { setType('expense');
setCategory('Food'); }} className={flex-1 py-1.5 text-sm font-medium
rounded-md transition-all ${type === 'expense' ? 'bg-white text-gray-
900 shadow' : 'text-gray-500 hover:text-gray-700'}}>Expense</button>
    <button type="button" onClick={() => { setType('income');
setCategory('Allowance'); }} className={flex-1 py-1.5 text-sm font-
medium rounded-md transition-all ${type === 'income' ? 'bg-white
text-gray-900 shadow' : 'text-gray-500 hover:text-gray-
700'}}>Income</button>
  </div>

  <div className="grid grid-cols-2 gap-3">
    <div className="col-span-2">
```

```
      <input type="text" value={title} onChange={(e) =>
setTitle(e.target.value)} placeholder="Description (e.g. Library
Fine)" className="w-full px-3 py-2 text-sm bg-gray-50 border border-
gray-200 rounded focus:outline-none focus:ring-1 focus:ring-indigo-
500" />
    </div>
    <div>
      <input type="number" value={amount} onChange={(e) =>
setAmount(e.target.value)} placeholder="Amount (₹)" className="w-full
px-3 py-2 text-sm bg-gray-50 border border-gray-200 rounded
focus:outline-none focus:ring-1 focus:ring-indigo-500" />
    </div>
    <div>
      <input type="date" value={date} onChange={(e) =>
setDate(e.target.value)} className="w-full px-3 py-2 text-sm bg-gray-
50 border border-gray-200 rounded focus:outline-none focus:ring-1
focus:ring-indigo-500 text-gray-600" />
    </div>
  </div>

  <div>
    <select value={category} onChange={(e) =>
setCategory(e.target.value)} className="w-full px-3 py-2 text-sm bg-
gray-50 border border-gray-200 rounded focus:outline-none focus:ring-
1 focus:ring-indigo-500 text-gray-700">
      {type === 'expense'
        ? expenseCategories.map(cat => <option key={cat}
value={cat}>{cat}</option>)
        : incomeCategories.map(cat => <option key={cat}
value={cat}>{cat}</option>)
      }
    </select>
  </div>

  <button type="submit" className="w-full flex items-center justify-
center gap-2 py-2 bg-indigo-600 text-white rounded font-medium
hover:bg-indigo-700 transition-colors text-sm">
    <IconPlusCircle className="w-4 h-4" /> Add Record
```

```
  </button>
</form>

); }
// --- HISTORY / LIST COMPONENT --- // Filters removed for Review 1
function History({ transactions, onDelete }) { const getCategoryIcon
= (category, type) => { if (type === 'income') return ; switch
(category) { case 'Food': return ; case 'Travel': return ; case
'Academics': return ; case 'Entertainment': return ; default: return
; } };
return (
```

Transaction History

Showing all records (Filter logic pending Review 2).

```
 <div className="bg-white rounded-2xl shadow-sm border border-gray-
100 overflow-hidden">
    {transactions.length === 0 ? (
      <div className="p-8 text-center text-gray-500 flex flex-col
items-center">
        <IconList className="w-12 h-12 text-gray-300 mb-3" />
        <p className="font-medium">No transactions found.</p>
      </div>
    ) : (
      <ul className="divide-y divide-gray-100">
        {transactions.map(txn => (
          <li key={txn.id} className="p-4 sm:p-6 hover:bg-gray-50
transition-colors flex items-center justify-between group">
            <div className="flex items-center gap-4">
              <div className={p-3 rounded-full ${txn.type ===
'income' ? 'bg-emerald-100' : 'bg-gray-100'}}>
                {getCategoryIcon(txn.category, txn.type)}
              </div>
              <div>
```

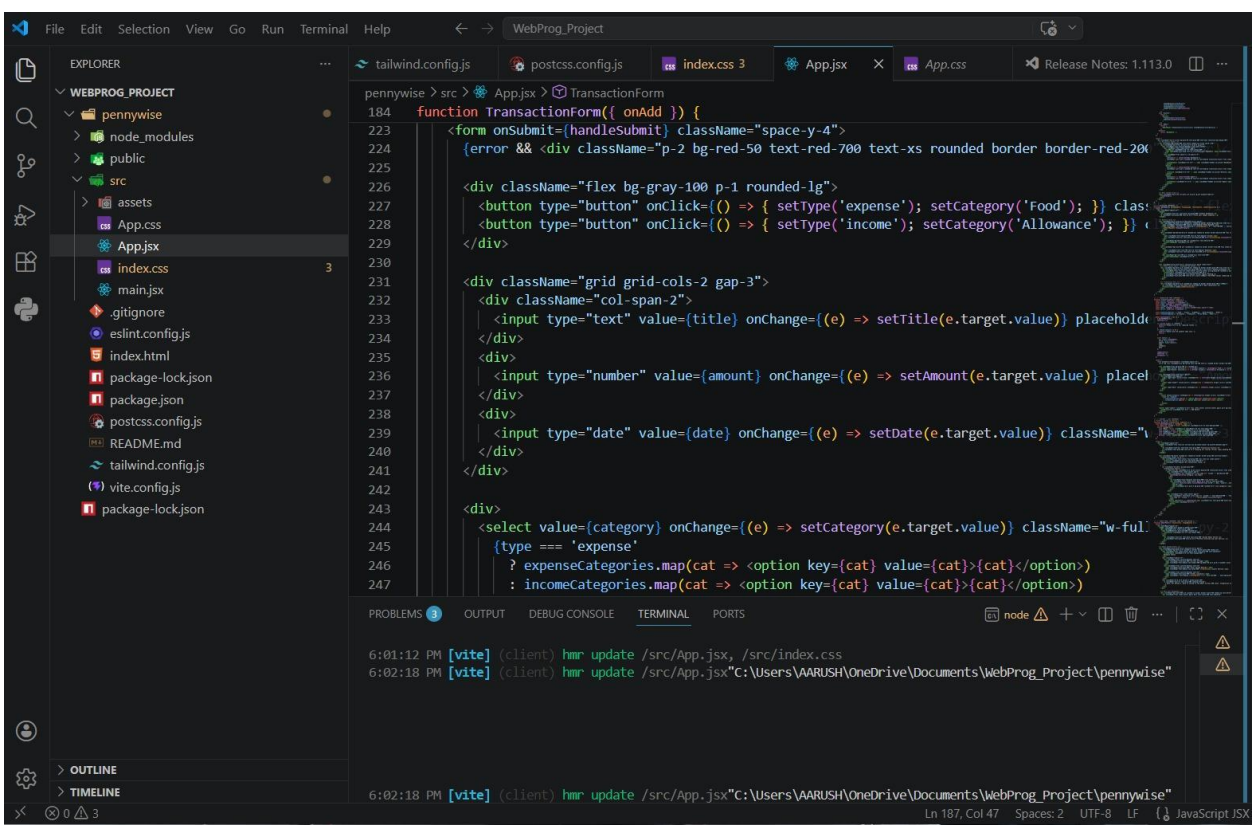
```
                <h3 className="font-semibold text-gray-
900">{txn.title}</h3>
                <div className="flex items-center gap-2 mt-1 text-xs
text-gray-500">
                  <span>{new Date(txn.date).toLocaleDateString('en-
GB', { day: 'numeric', month: 'short', year: 'numeric' })}</span>
                  <span>•</span>
                  <span className="px-2 py-0.5 bg-gray-100 rounded-
full">{txn.category}</span>
                </div>
              </div>
            </div>

            <div className="flex items-center gap-4">
              <span className={font-bold ${txn.type === 'income' ?
'text-emerald-600' : 'text-gray-900'}}>
                {txn.type === 'income' ? '+' : '-
'}₹{txn.amount.toLocaleString()}
              </span>
              <button onClick={() => onDelete(txn.id)} className="p-2
text-gray-400 hover:text-red-500 hover:bg-red-50 rounded-lg
transition-colors opacity-100 sm:opacity-0 sm:group-hover:opacity-100
focus:opacity-100" title="Delete Transaction">
                <IconTrash2 className="w-4 h-4" />
              </button>
            </div>
          </li>
        ))}
      </ul>
    )}
  </div>
</div>

); }
// --- ADMIN PANEL COMPONENT (NEW MVP FOR REVIEW 1) --- function
AdminPanel({ transactions, totalBalance }) { return (
```

System Admin Portal

Platform overview and user metrics.

```
 {/* Admin System Status */}
  <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
    <div className="bg-white p-6 rounded-xl border border-gray-200 shadow-sm">
      <div className="flex items-center gap-2 mb-4 text-gray-700 font-semibold">
        <IconDatabase className="w-5 h-5 text-indigo-500" />
        Database Status
      </div>
      <div className="space-y-3">
        <div className="flex justify-between text-sm">
          <span className="text-gray-500">Current Storage:</span>
          <span className="font-medium text-amber-600 bg-amber-50 px-2 py-0.5
rounded">Local State (Temporary)</span>
        </div>
        <div className="flex justify-between text-sm">
          <span className="text-gray-500">Total System Records:</span>
          <span className="font-medium text-gray-900">{transactions.length}
Entries</span>
        </div>
        <div className="flex justify-between text-sm">
          <span className="text-gray-500">Global Liquidity:</span>
          <span className={font-medium ${totalBalance < 0 ? 'text-red-600' : 'text-
emerald-600'}}>₹{totalBalance.toLocaleString()}</span>
        </div>
      </div>
      <div className="mt-6 pt-4 border-t border-gray-100">
        <p className="text-xs text-gray-400 leading-relaxed">
          * Note: For Review 1, data is stored in the React Virtual DOM state.
Integration with a persistent backend database / local storage is scheduled for
Review 2.
        </p>
      </div>
    </div>

    {/* Mock User Management */}
    <div className="bg-slate-800 p-6 rounded-xl border border-slate-700 shadow-sm
text-white">
      <div className="flex items-center gap-2 mb-4 text-slate-200 font-semibold">
        <IconList className="w-5 h-5 text-amber-400" />
```

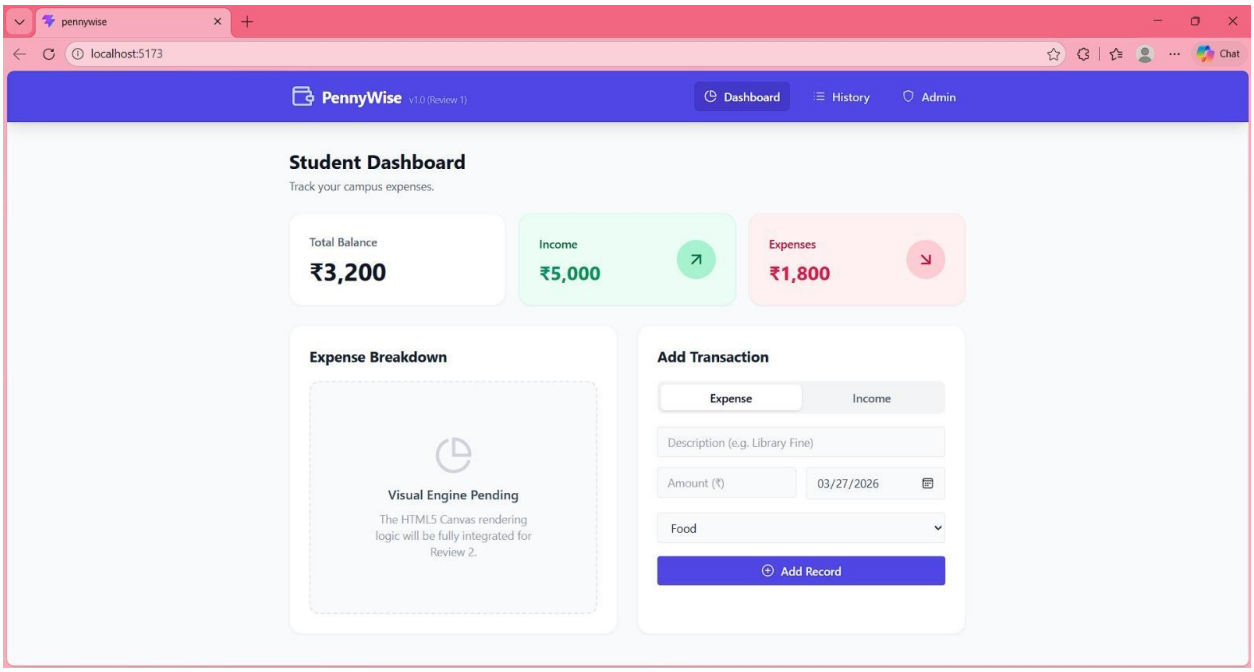
```
        Registered Users
      </div>
      <div className="bg-slate-900/50 rounded-lg p-3 border border-slate-700 mb-3
flex justify-between items-center">
        <div className="flex items-center gap-3">
          <div className="w-8 h-8 rounded-full bg-indigo-500 flex items-center
justify-center font-bold text-sm">AS</div>
          <div>
            <div className="text-sm font-medium">Aarush Singh</div>
            <div className="text-xs text-slate-400">VIT Vellore • Active</div>
          </div>
        </div>
        <div className="text-xs font-mono text-emerald-400 bg-emerald-400/10 px-2
py-1 rounded">Online</div>
      </div>

      <button disabled className="w-full py-2 mt-4 bg-slate-700 text-slate-400
rounded-lg text-sm font-medium cursor-not-allowed border border-slate-600">
        Add New User (Locked - Review 2)
      </button>
    </div>
  </div>
</div>

); }
```



The above VS CODE screenshot depicts the different libraries used for all the current functionalities in the website (during Review 1).

Additional libraries will be added into the same for the complete implementation of PennyWise along with a fully functional admin page.

Code Implementation -

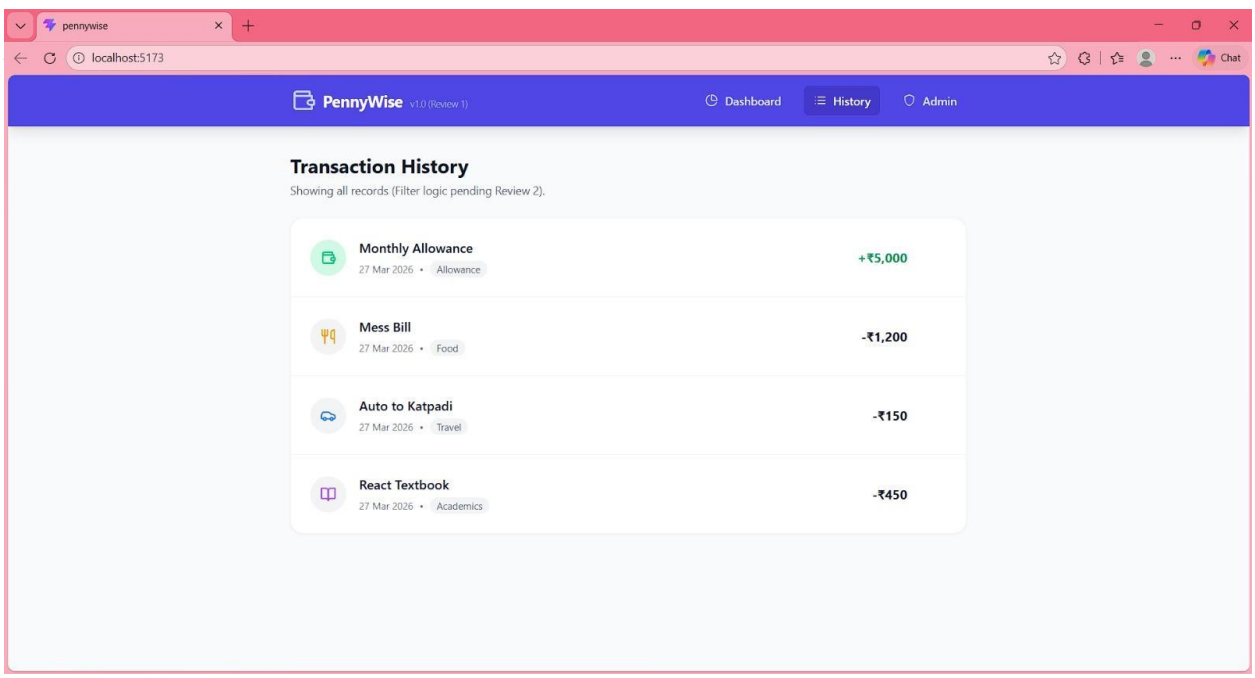


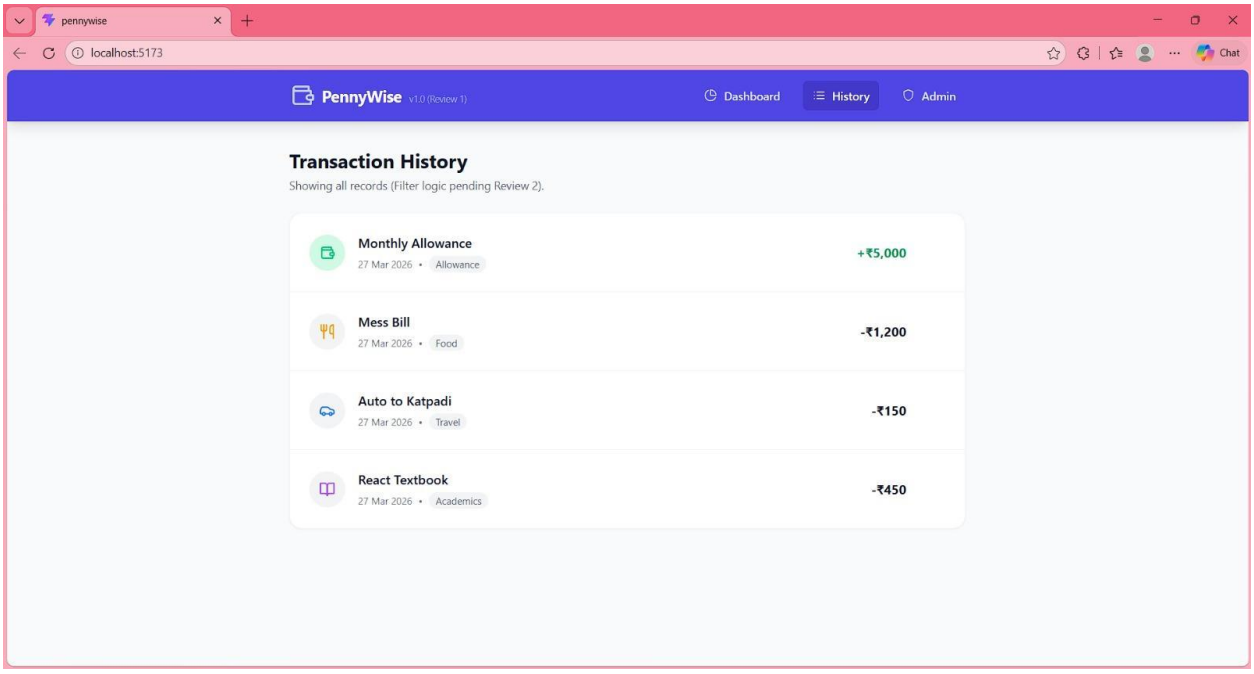
1. Student Dashboard

The Student Dashboard serves as the primary interface for users to monitor their financial activity. It dynamically displays Total Balance, Income, and Expenses using React's `useMemo` hook for efficient recalculation based on transaction data. The layout includes three summary cards and two sections: an Expense Breakdown placeholder (to be implemented in Review 2 using HTML5 Canvas) and an Add Transaction form. The form captures user input, validates required fields, and updates the application state instantly upon submission. This ensures real-time updates without requiring page reloads.

Implementation Snippet:

```
<Dashboard
 totalBalance={totalBalance}
  totalIncome={totalIncome}
  totalExpense={totalExpense}
  onAddTransaction={addTransaction}
```



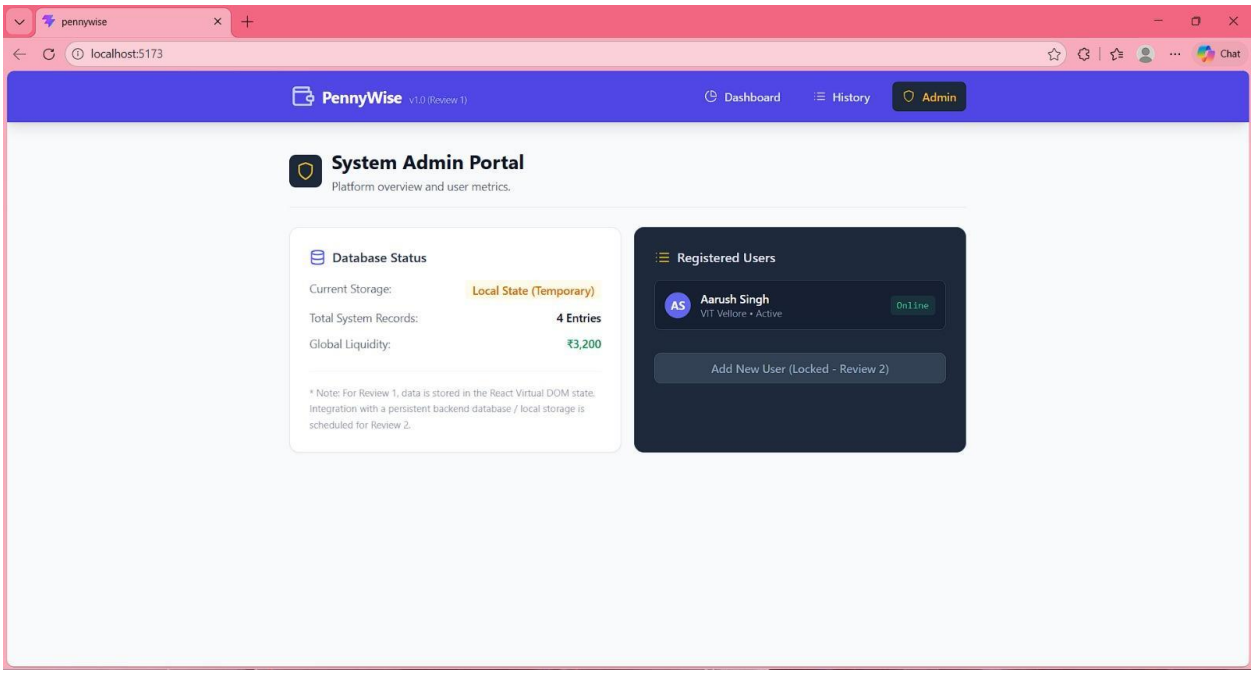


2. Transaction History

The Transaction History screen provides a detailed list of all recorded financial transactions. Each entry displays the title, date, category, and amount, with conditional formatting applied to distinguish income and expenses. The data is rendered by iterating over the transactions array. A delete functionality is included, allowing users to remove specific records, which updates the UI instantly. Advanced features such as filtering and sorting are intentionally deferred to Review 2, as indicated in the interface.

Implementation Snippet:

```
<History
 transactions={transactions}
  onDelete={deleteTransaction}
/>
```



3. System Admin Portal

The System Admin Portal is a basic administrative interface designed for Review 1. It provides an overview of system-level metrics, including current storage type (local state), total number of records, and global financial balance. Additionally, a mock “Registered Users” section is included to represent future user management functionality. Interactive features such as adding new users are disabled and planned for implementation in Review 2, demonstrating forward scalability of the system.

Implementation Snippet:

```
<AdminPanel
 transactions={transactions}
  totalBalance={totalBalance}
/>
```

4. Navigation and Routing Logic

The application uses a simple state-based routing mechanism controlled by the currentTab variable. Based on the selected tab (Dashboard, History, or Admin), the corresponding component is rendered dynamically. This approach keeps the implementation lightweight and suitable for a Minimum Viable Product (MVP).

Implementation Snippet:

```
switch(currentTab) {
  case 'dashboard': return <Dashboard ... />;
  case 'history': return <History ... />;
  case 'admin': return <AdminPanel ... />;
}
```